



Paris 1 Panthéon-Sorbonne University
U.F.R. 27 – Mathematics and Computer Science

PROJECT REPORT
MASTER 2 MMMEF
MATHEMATICAL METHODS IN ECONOMICS AND FINANCE

TOPIC

Quantitative Finance Project: Mean–Variance Portfolio Optimization

Written by:

- Marc SU

Supervised by:

- Professor Pierre Gruet

Academic year 2025 – 2026

Acknowledgements

I would like to sincerely thank Professor Pierre Gruet for his numerical methods course, as well as for the project topic. His teaching helped me deepen my understanding of optimization methods and better understand their relevance in a quantitative finance framework.

I would also like to thank him for the guidance provided throughout the semester, which helped me structure this work and connect the theoretical aspects of numerical methods with their practical implementation in Python.

Introduction

Portfolio optimization is a central topic in quantitative finance. The main objective is to determine how an investor should allocate wealth among several risky assets in order to obtain a satisfactory trade-off between expected return and risk. In the classical mean-variance framework, risk is measured by the variance of the portfolio return, while performance is measured by the expected return.

In this project, we study a mean-variance portfolio allocation problem. The objective function combines a quadratic risk term and a linear expected-return term. The balance between these two components is governed by a risk-aversion parameter $\phi > 0$. The larger this parameter is, the more importance is given to the expected return in the optimization criterion.

The original problem is constrained by two natural portfolio restrictions. First, the sum of the portfolio weights must be equal to one, meaning that the whole capital is invested. Second, the weights must be nonnegative, which corresponds to a no-short-selling constraint. In the first part of the project, the equality constraint is eliminated in order to obtain a reduced problem in $\mathbb{R}^{n-1}$. Then, the inequality constraints are temporarily removed, leading to an unconstrained quadratic minimization problem.

Several numerical optimization methods are then implemented and compared: steepest descent with Wolfe-compliant step lengths, Newton's method, the linear conjugate gradient method, and optimization routines from `scipy.optimize`. These methods are applied to the same reduced quadratic objective, which allows us to compare their convergence behavior and numerical efficiency.

Finally, the no-short-selling constraints are enforced again. The constrained problem is solved as a quadratic program on the simplex. This last part highlights the difference between the unconstrained solution, where some weights may be negative, and the constrained solution, which must remain feasible as a portfolio allocation.

The Python code used in this report is included in Appendix 4.4. Some places are also left in the document for figures generated from Python.

Contents

Acknowledgements	1
Introduction	1
1 Preliminary Theory and Applications	4
1.1 Mathematical background	4
1.2 Numerical methods used in the report	6
2 Mean–Variance Portfolio Optimization Problem	8
2.1 Portfolio return	8
2.2 Mean–variance objective	9
2.3 Reduction to a $(n - 1)$ -dimensional Problem	10
2.4 Eliminating the equality constraint	10
2.5 Reduced expected return	11
2.6 Reduced variance	11
2.7 Final reduced objective	12
3 Numerical Methods for the Unconstrained Reduced Problem	13
3.1 Steepest Descent with Wolfe-Compliant Step Lengths	13
3.2 Algorithm	13
3.3 Step length	13
3.4 Initialization and stopping rule	14
3.5 Numerical result for $n = 12$ and $\phi = 5$	14
3.6 Newton Method with Wolfe Step Lengths	15
3.7 Newton direction	16
3.8 Numerical result for $n = 12$ and $\phi = 5$	16
3.9 Linear Conjugate Gradient	17
3.10 Linear system formulation	17
3.11 Conjugate gradient algorithm	17
3.12 Numerical result for $n = 12$ and $\phi = 5$	18
3.13 Solution Using an Optimization Library	19
3.14 Reference solution	19
3.15 SciPy BFGS	20
3.16 SciPy Newton-CG	20
3.17 Discussion	20

4	Constrained Portfolio Optimization	21
4.1	Simplex-constrained problem	21
4.2	KKT system on the free set	21
4.3	SLSQP implementation	22
4.4	Numerical optimum for $n = 12$ and $\phi = 5$	22
	Conclusion	24
	Appendix: Python Code	25

Chapter 1: Preliminary Theory and Applications

1.1 Mathematical background

In this first chapter, we recall several definitions and mathematical results that will be used throughout the report. These elements provide a rigorous foundation for the numerical methods applied to the portfolio optimization problem.

Definition 1.1 (Financial portfolio). A financial portfolio is a vector

$$x = (x_1, \dots, x_n)^\top \in \mathbb{R}^n,$$

where x_i represents the proportion of wealth invested in asset i . When

$$\sum_{i=1}^n x_i = 1,$$

the portfolio is fully invested. If, in addition, $x_i \geq 0$ for all i , then short-selling is not allowed.

Definition 1.2 (Expected return). Let R_i be the random return of asset i . The expected return of this asset is defined by

$$\mu_i = \mathbb{E}(R_i).$$

For a portfolio $x \in \mathbb{R}^n$, the expected portfolio return is

$$\mathbb{E}(R) = \sum_{i=1}^n x_i \mathbb{E}(R_i) = \mu^\top x.$$

Definition 1.3 (Covariance matrix). The covariance matrix of the asset returns is the matrix

$$\Sigma = (\Sigma_{ij})_{1 \leq i, j \leq n},$$

defined by

$$\Sigma_{ij} = \text{Cov}(R_i, R_j).$$

It measures the dependence between the returns of the different assets.

Proposition 1.1 (Variance of a portfolio). *Let $x \in \mathbb{R}^n$ be a vector of portfolio weights. The variance of the portfolio return is given by*

$$\text{Var}(R) = x^\top \Sigma x.$$

Proof. The portfolio return is

$$R = \sum_{i=1}^n x_i R_i.$$

By bilinearity of the covariance operator, we have

$$\text{Var}(R) = \text{Cov} \left(\sum_{i=1}^n x_i R_i, \sum_{j=1}^n x_j R_j \right).$$

Therefore,

$$\text{Var}(R) = \sum_{i=1}^n \sum_{j=1}^n x_i x_j \text{Cov}(R_i, R_j).$$

Since $\Sigma_{ij} = \text{Cov}(R_i, R_j)$, it follows that

$$\text{Var}(R) = \sum_{i=1}^n \sum_{j=1}^n x_i x_j \Sigma_{ij} = x^\top \Sigma x.$$

□

Definition 1.4 (Quadratic optimization problem). A quadratic optimization problem is an optimization problem whose objective function is quadratic, that is, a function of the form

$$f(x) = c + a^\top x + x^\top Bx,$$

where $c \in \mathbb{R}$, $a \in \mathbb{R}^d$ and $B \in \mathbb{R}^{d \times d}$.

Proposition 1.2 (Gradient and Hessian of a quadratic function). *Let*

$$f(x) = c + a^\top x + x^\top Bx,$$

where B is symmetric. Then

$$\nabla f(x) = a + 2Bx,$$

and

$$\nabla^2 f(x) = 2B.$$

Proof. We differentiate each term separately. The constant term c has zero derivative. The linear term $a^\top x$ has gradient a . Since B is symmetric,

$$\nabla(x^\top Bx) = 2Bx.$$

Therefore,

$$\nabla f(x) = a + 2Bx.$$

Differentiating once more gives

$$\nabla^2 f(x) = 2B.$$

□

Definition 1.5 (Convexity). A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is said to be convex if, for all $x, y \in \mathbb{R}^d$ and for all $\lambda \in [0, 1]$,

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y).$$

Theorem 1.1 (Convexity condition for a twice differentiable function). *Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a twice differentiable function. If*

$$\nabla^2 f(x)$$

is positive semidefinite for every $x \in \mathbb{R}^d$, then f is convex.

Remark 1.1. In this project, the objective function is quadratic. Therefore, its Hessian is constant. Studying convexity is thus equivalent to studying the sign of the matrix associated with the quadratic part.

1.2 Numerical methods used in the report

In this report, we use several classical numerical methods to solve the reduced problem. The objective is to compare their behavior in terms of convergence and number of iterations.

Definition 1.6 (Descent method). A descent method is an iterative method of the form

$$x_{k+1} = x_k + \alpha_k p_k,$$

where p_k is a descent direction and $\alpha_k > 0$ is a step size.

Definition 1.7 (Descent direction). Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable. A vector p_k is a descent direction at x_k if

$$p_k^\top \nabla f(x_k) < 0.$$

Proposition 1.3 (The opposite gradient is a descent direction). *Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be differentiable. If $\nabla f(x_k) \neq 0$, then*

$$p_k = -\nabla f(x_k)$$

is a descent direction.

Proof. We compute

$$p_k^\top \nabla f(x_k) = (-\nabla f(x_k))^\top \nabla f(x_k).$$

Therefore,

$$p_k^\top \nabla f(x_k) = -\|\nabla f(x_k)\|^2.$$

If $\nabla f(x_k) \neq 0$, then

$$-\|\nabla f(x_k)\|^2 < 0.$$

Thus, p_k is a descent direction. □

Definition 1.8 (Wolfe conditions). Let f be a differentiable function and let p_k be a descent direction at x_k . A step size $\alpha_k > 0$ satisfies the Wolfe conditions if there exist constants $0 < c_1 < c_2 < 1$ such that

$$f(x_k + \alpha_k p_k) \leq f(x_k) + c_1 \alpha_k \nabla f(x_k)^\top p_k,$$

and

$$\nabla f(x_k + \alpha_k p_k)^\top p_k \geq c_2 \nabla f(x_k)^\top p_k.$$

Remark 1.2. The Wolfe conditions are used to choose a step size that ensures a sufficient decrease of the objective function while avoiding excessively small steps.

Theorem 1.2 (Newton convergence for a quadratic function). *Let f be a strictly convex quadratic function of the form*

$$f(x) = c + a^\top x + x^\top Bx,$$

where B is symmetric positive definite, so that the minimizer is unique. Then Newton's method converges to the minimizer in one iteration if the step size $\alpha_k = 1$ is accepted.

Proof. The minimizer x^* satisfies

$$\nabla f(x^*) = 0.$$

Since

$$\nabla f(x) = a + 2Bx,$$

we have

$$a + 2Bx^* = 0.$$

The Newton direction p_k satisfies

$$\nabla^2 f(x_k)p_k = -\nabla f(x_k).$$

Since $\nabla^2 f(x_k) = 2B$, we obtain

$$2Bp_k = -(a + 2Bx_k).$$

Using $a = -2Bx^*$, this becomes

$$2Bp_k = 2B(x^* - x_k).$$

Since B is invertible, we get

$$p_k = x^* - x_k.$$

Thus, with $\alpha_k = 1$,

$$x_{k+1} = x_k + p_k = x^*.$$

□

Definition 1.9 (Conjugate gradient method). The conjugate gradient method is an iterative method used to solve a linear system

$$Ax = b,$$

where A is symmetric positive definite. It constructs conjugate directions that allow the exact solution to be reached in a finite number of iterations in exact arithmetic.

Theorem 1.3 (Finite convergence of conjugate gradient). *Let $A \in \mathbb{R}^{d \times d}$ be a symmetric positive definite matrix. In exact arithmetic, the conjugate gradient method applied to the system*

$$Ax = b$$

converges in at most d iterations.

Remark 1.3. In this project, the dimension of the reduced problem is $n - 1$. For $n = 12$, the reduced dimension is therefore 11. In theory, conjugate gradient can therefore converge in at most 11 iterations. In practice, rounding errors may lead to a few additional iterations.

Chapter 2: Mean–Variance Portfolio Optimization Problem

2.1 Portfolio return

Let R_i be the return of asset i , for $i = 1, \dots, n$. We denote

$$\mu_i = \mathbb{E}(R_i), \quad \Sigma_{ij} = \text{Cov}(R_i, R_j).$$

Definition 2.1 (Portfolio weights). The vector

$$x = (x_1, \dots, x_n)^\top \in \mathbb{R}^n$$

is called the vector of portfolio weights. Each coefficient x_i represents the proportion of wealth invested in asset i .

Definition 2.2 (Admissible portfolio). A portfolio is admissible without short-selling if

$$x_i \geq 0, \quad i = 1, \dots, n,$$

and

$$\sum_{i=1}^n x_i = 1.$$

The first condition means that no asset can have a negative weight. The second condition means that the whole wealth is invested.

The portfolio return is

$$R = \sum_{i=1}^n x_i R_i.$$

Proposition 2.1 (Expected return of the portfolio). *The expected return of the portfolio is*

$$\mathbb{E}(R) = \mu^\top x.$$

Proof. By linearity of the expectation,

$$\mathbb{E}(R) = \mathbb{E}\left(\sum_{i=1}^n x_i R_i\right) = \sum_{i=1}^n x_i \mathbb{E}(R_i).$$

Since $\mu_i = \mathbb{E}(R_i)$, we obtain

$$\mathbb{E}(R) = \sum_{i=1}^n x_i \mu_i = \mu^\top x.$$

□

Proposition 2.2 (Portfolio variance). *The variance of the portfolio return is*

$$\text{Var}(R) = x^\top \Sigma x.$$

Proof. Using the bilinearity of covariance,

$$\text{Var}(R) = \text{Cov} \left(\sum_{i=1}^n x_i R_i, \sum_{j=1}^n x_j R_j \right).$$

Therefore,

$$\text{Var}(R) = \sum_{i=1}^n \sum_{j=1}^n x_i x_j \text{Cov}(R_i, R_j).$$

Since $\Sigma_{ij} = \text{Cov}(R_i, R_j)$, we get

$$\text{Var}(R) = \sum_{i=1}^n \sum_{j=1}^n x_i x_j \Sigma_{ij} = x^\top \Sigma x.$$

□

2.2 Mean–variance objective

Definition 2.3 (Mean–variance objective). For a risk-aversion parameter $\phi > 0$, the mean–variance objective is defined by

$$J(x) = x^\top \Sigma x - \phi \mu^\top x.$$

The first term measures the risk of the portfolio, while the second term rewards expected return.

Remark 2.1. The parameter ϕ controls the trade-off between expected return and variance. A larger value of ϕ gives more importance to expected return, whereas a smaller value gives more importance to risk reduction.

The constrained portfolio optimization problem is

$$\min_{x \in \mathbb{R}^n} J(x) \quad \text{subject to} \quad x_i \geq 0 \ (i = 1, \dots, n), \quad \sum_{i=1}^n x_i = 1.$$

Proposition 2.3 (Quadratic nature of the objective). *The function*

$$J(x) = x^\top \Sigma x - \phi \mu^\top x$$

is a quadratic function of x .

Proof. The term $x^\top \Sigma x$ is quadratic in x , while the term $-\phi \mu^\top x$ is linear in x . Therefore, their sum is a quadratic function. □

Proposition 2.4 (Gradient and Hessian of the objective). *The gradient and Hessian of J are*

$$\nabla J(x) = 2\Sigma x - \phi \mu,$$

and

$$\nabla^2 J(x) = 2\Sigma.$$

Proof. Since Σ is a covariance matrix, it is symmetric. Hence,

$$\nabla(x^\top \Sigma x) = 2\Sigma x.$$

Also,

$$\nabla(-\phi\mu^\top x) = -\phi\mu.$$

Thus,

$$\nabla J(x) = 2\Sigma x - \phi\mu.$$

Differentiating once again gives

$$\nabla^2 J(x) = 2\Sigma.$$

□

2.3 Reduction to a $(n - 1)$ -dimensional Problem

Proposition 2.5 (Elimination of the equality constraint). *The equality constraint*

$$\sum_{i=1}^n x_i = 1$$

allows us to reduce the original n -dimensional problem to a problem in dimension $n - 1$.

Proof. From

$$\sum_{i=1}^n x_i = 1,$$

we can write

$$x_n = 1 - \sum_{i=1}^{n-1} x_i.$$

Therefore, once $x_1, \dots, x_{n-1}$ are known, the last coordinate x_n is uniquely determined. Hence the problem can be rewritten using only $n - 1$ variables. □

2.4 Eliminating the equality constraint

Using the equality constraint,

$$\sum_{i=1}^n x_i = 1,$$

we eliminate the last coordinate:

$$x_n = 1 - \sum_{i=1}^{n-1} x_i.$$

The reduced variable is still denoted by

$$x = (x_1, \dots, x_{n-1}) \in \mathbb{R}^{n-1}.$$

The inequality constraints become

$$x_1 \geq 0, \dots, x_{n-1} \geq 0, \quad 1 - \sum_{i=1}^{n-1} x_i \geq 0.$$

2.5 Reduced expected return

Substituting $x_n = 1 - \sum_{i=1}^{n-1} x_i$ gives

$$\begin{aligned}\mathbb{E}(R) &= \sum_{i=1}^{n-1} x_i \mathbb{E}(R_i) + \left(1 - \sum_{i=1}^{n-1} x_i\right) \mathbb{E}(R_n) \\ &= \mathbb{E}(R_n) + \sum_{i=1}^{n-1} x_i (\mathbb{E}(R_i) - \mathbb{E}(R_n)).\end{aligned}$$

2.6 Reduced variance

We start from

$$\text{Var}(R) = \sum_{i=1}^n \sum_{j=1}^n x_i x_j \text{Cov}(R_i, R_j).$$

Separating the terms involving the index n gives

$$\begin{aligned}\text{Var}(R) &= \sum_{i=1}^{n-1} \sum_{j=1}^{n-1} x_i x_j \text{Cov}(R_i, R_j) + 2 \sum_{i=1}^{n-1} x_i x_n \text{Cov}(R_i, R_n) \\ &\quad + x_n^2 \text{Cov}(R_n, R_n).\end{aligned}$$

For the cross term,

$$x_i x_n = x_i \left(1 - \sum_{k=1}^{n-1} x_k\right) = x_i - \sum_{k=1}^{n-1} x_i x_k,$$

so

$$2 \sum_{i=1}^{n-1} x_i x_n \text{Cov}(R_i, R_n) = 2 \sum_{i=1}^{n-1} x_i \text{Cov}(R_i, R_n) - 2 \sum_{i=1}^{n-1} \sum_{k=1}^{n-1} x_i x_k \text{Cov}(R_i, R_n).$$

For the square term,

$$x_n^2 = \left(1 - \sum_{k=1}^{n-1} x_k\right)^2 = 1 - 2 \sum_{k=1}^{n-1} x_k + \sum_{k=1}^{n-1} \sum_{\ell=1}^{n-1} x_k x_\ell.$$

Thus,

$$x_n^2 \text{Cov}(R_n, R_n) = \text{Cov}(R_n, R_n) - 2 \sum_{k=1}^{n-1} x_k \text{Cov}(R_n, R_n) + \sum_{k=1}^{n-1} \sum_{\ell=1}^{n-1} x_k x_\ell \text{Cov}(R_n, R_n).$$

Collecting constant, linear and quadratic terms yields

$$\begin{aligned}\text{Var}(R) &= \text{Cov}(R_n, R_n) \\ &\quad + \sum_{i=1}^{n-1} x_i \left[-2 \text{Cov}(R_n, R_n) + 2 \text{Cov}(R_i, R_n)\right] \\ &\quad + \sum_{i=1}^{n-1} \sum_{j=1}^{n-1} x_i x_j \left[\text{Cov}(R_i, R_j) - \text{Cov}(R_i, R_n) - \text{Cov}(R_j, R_n) + \text{Cov}(R_n, R_n)\right].\end{aligned}$$

Proposition 2.6 (Reduced variance formula). *After eliminating x_n , the variance can be written as a quadratic function of the reduced variable*

$$x = (x_1, \dots, x_{n-1}) \in \mathbb{R}^{n-1}.$$

More precisely,

$$\begin{aligned} \text{Var}(R) &= \text{Cov}(R_n, R_n) \\ &+ \sum_{i=1}^{n-1} x_i \left[-2\text{Cov}(R_n, R_n) + 2\text{Cov}(R_i, R_n) \right] \\ &+ \sum_{i=1}^{n-1} \sum_{j=1}^{n-1} x_i x_j \left[\text{Cov}(R_i, R_j) - \text{Cov}(R_i, R_n) - \text{Cov}(R_j, R_n) + \text{Cov}(R_n, R_n) \right]. \end{aligned}$$

2.7 Final reduced objective

Using

$$J(x) = -\phi \mathbb{E}(R) + \text{Var}(R),$$

we obtain the reduced objective $\tilde{J} : \mathbb{R}^{n-1} \rightarrow \mathbb{R}$:

$$\begin{aligned} \tilde{J}(x) &= -\phi \mathbb{E}(R_n) + \text{Cov}(R_n, R_n) \\ &+ \sum_{i=1}^{n-1} x_i \left[-\phi (\mathbb{E}(R_i) - \mathbb{E}(R_n)) - 2\text{Cov}(R_n, R_n) + 2\text{Cov}(R_i, R_n) \right] \\ &+ \sum_{i=1}^{n-1} \sum_{j=1}^{n-1} x_i x_j \left[\text{Cov}(R_i, R_j) - \text{Cov}(R_i, R_n) - \text{Cov}(R_j, R_n) + \text{Cov}(R_n, R_n) \right]. \end{aligned}$$

The reduced constrained problem is therefore

$$\min_{x \in \mathbb{R}^{n-1}} \tilde{J}(x) \quad \text{subject to} \quad x_i \geq 0 \quad (i = 1, \dots, n-1), \quad 1 - \sum_{i=1}^{n-1} x_i \geq 0.$$

Chapter 3: Numerical Methods for the Unconstrained Reduced Problem

In Questions 2–5, the inequality constraints are removed. We solve

$$\min_{x \in \mathbb{R}^{n-1}} \tilde{J}(x).$$

Since $\tilde{J}$ is quadratic, it can be written as

$$\tilde{J}(x) = c + a^\top x + x^\top Bx,$$

where B is symmetric. Hence,

$$\nabla \tilde{J}(x) = a + 2Bx, \quad \nabla^2 \tilde{J}(x) = 2B.$$

3.1 Steepest Descent with Wolfe-Compliant Step Lengths

3.2 Algorithm

Starting from $x_0 \in \mathbb{R}^{n-1}$, the steepest descent method generates a sequence (x_k) defined by

$$p_k = -\nabla \tilde{J}(x_k), \quad x_{k+1} = x_k + \alpha_k p_k.$$

Proposition 3.1 (Steepest descent direction). *At each iteration, if*

$$\nabla \tilde{J}(x_k) \neq 0,$$

then

$$p_k = -\nabla \tilde{J}(x_k)$$

is a descent direction.

Proof. We compute

$$p_k^\top \nabla \tilde{J}(x_k) = -\nabla \tilde{J}(x_k)^\top \nabla \tilde{J}(x_k) = -\|\nabla \tilde{J}(x_k)\|^2.$$

Since $\nabla \tilde{J}(x_k) \neq 0$, we have

$$-\|\nabla \tilde{J}(x_k)\|^2 < 0.$$

Therefore, p_k is a descent direction. □

3.3 Step length

Proposition 3.2 (Exact line-search step). *For the quadratic objective $\tilde{J}$, the exact line-search step along the steepest descent direction is*

$$\alpha_k^* = \frac{\nabla \tilde{J}(x_k)^\top \nabla \tilde{J}(x_k)}{\nabla \tilde{J}(x_k)^\top (\nabla^2 \tilde{J}) \nabla \tilde{J}(x_k)}.$$

Proof. Let

$$\varphi(\alpha) = \tilde{J}(x_k + \alpha p_k).$$

The exact step minimizes φ , so it satisfies

$$\varphi'(\alpha) = 0.$$

Since $\tilde{J}$ is quadratic, φ is a one-dimensional quadratic function. Using $p_k = -\nabla \tilde{J}(x_k)$, solving $\varphi'(\alpha) = 0$ gives

$$\alpha_k^* = \frac{\nabla \tilde{J}(x_k)^\top \nabla \tilde{J}(x_k)}{\nabla \tilde{J}(x_k)^\top (\nabla^2 \tilde{J}) \nabla \tilde{J}(x_k)}.$$

□

Remark 3.1. For a strictly convex quadratic objective, this exact line-search step satisfies the Wolfe conditions.

3.4 Initialization and stopping rule

The reduced initial point is the uniform portfolio without its last coordinate:

$$x_0 = \left(\frac{1}{n}, \dots, \frac{1}{n} \right) \in \mathbb{R}^{n-1}.$$

The stopping criterion is

$$\|\nabla \tilde{J}(x_k)\| \leq 10^{-5}.$$

3.5 Numerical result for $n = 12$ and $\phi = 5$

The Python implementation gives:

- iterations: 2789,
- final gradient norm: approximately 9.984714×10^{-6} ,
- objective value: $\tilde{J}(x^*) \approx 3.041284972991$.

The unconstrained minimizer is approximately

$$x^* = \begin{pmatrix} 0.26851017 \\ -0.60158666 \\ -0.18651313 \\ 0.95014284 \\ 0.05797520 \\ -0.01420054 \\ -0.39572754 \\ 0.18019927 \\ 0.24183310 \\ 0.41863451 \\ 0.06426101 \end{pmatrix}, \quad x_{12} = 1 - \sum_{i=1}^{11} x_i \approx 0.01647176.$$

Some components are negative because the inequality constraints are not enforced. This confirms that the unconstrained minimizer is not necessarily an admissible portfolio when short-selling is forbidden.

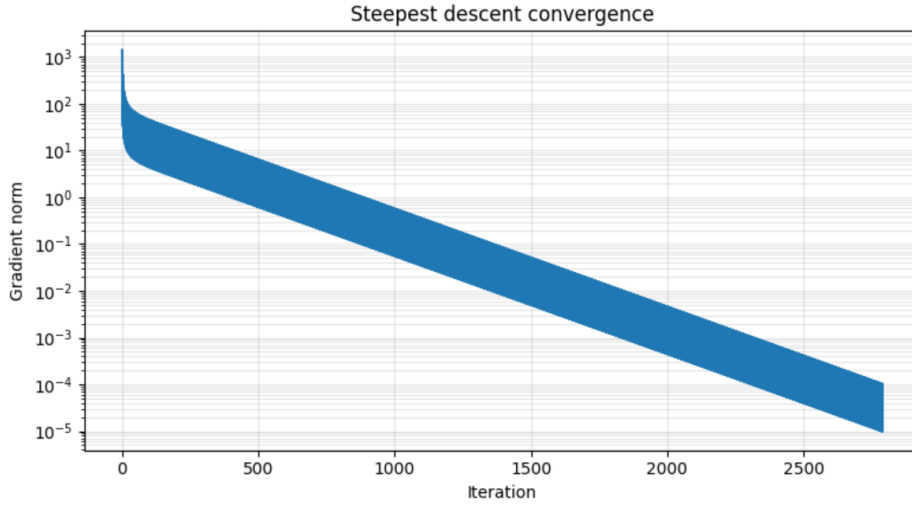


Figure 3.1: Convergence of the steepest descent method: gradient norm versus iteration.

3.6 Newton Method with Wolfe Step Lengths

Theorem 3.1 (One-step convergence of Newton's method). *For a strictly convex quadratic function, Newton's method converges in one iteration when the step size $\alpha_k = 1$ is used.*

Proof. The reduced objective can be written as

$$\tilde{J}(x) = c + a^\top x + x^\top Bx.$$

Its gradient and Hessian are

$$\nabla \tilde{J}(x) = a + 2Bx, \quad \nabla^2 \tilde{J}(x) = 2B.$$

The minimizer x^* satisfies

$$a + 2Bx^* = 0.$$

The Newton direction p_k is defined by

$$2Bp_k = -(a + 2Bx_k).$$

Since $a = -2Bx^*$, we obtain

$$2Bp_k = 2B(x^* - x_k).$$

Assuming B is invertible, this gives

$$p_k = x^* - x_k.$$

Therefore, with $\alpha_k = 1$,

$$x_{k+1} = x_k + p_k = x^*.$$

□

3.7 Newton direction

At iteration k , the Newton direction p_k solves

$$\nabla^2 \tilde{J}(x_k) p_k = -\nabla \tilde{J}(x_k),$$

that is

$$(2B)p_k = -(a + 2Bx_k).$$

The update is

$$x_{k+1} = x_k + \alpha_k p_k,$$

where α_k is chosen by a Wolfe line search.

Remark 3.2. Since the objective function is quadratic, its Hessian is constant. Therefore, Newton's method reaches the unconstrained minimizer in one iteration when the full step $\alpha_k = 1$ is accepted.

3.8 Numerical result for $n = 12$ and $\phi = 5$

The Python implementation gives:

- iterations: 1,
- objective value: $\tilde{J}(x^*) \approx 3.041284972984$.

The reduced minimizer is approximately

$$x^* = \begin{pmatrix} 0.26851035 \\ -0.60158785 \\ -0.18651334 \\ 0.95014403 \\ 0.05797526 \\ -0.01420058 \\ -0.39572781 \\ 0.18019933 \\ 0.24183317 \\ 0.41863455 \\ 0.06426108 \end{pmatrix}, \quad x_{12} \approx 0.01647181.$$

As expected for a quadratic objective, Newton's method reaches the minimizer in one iteration up to numerical precision.

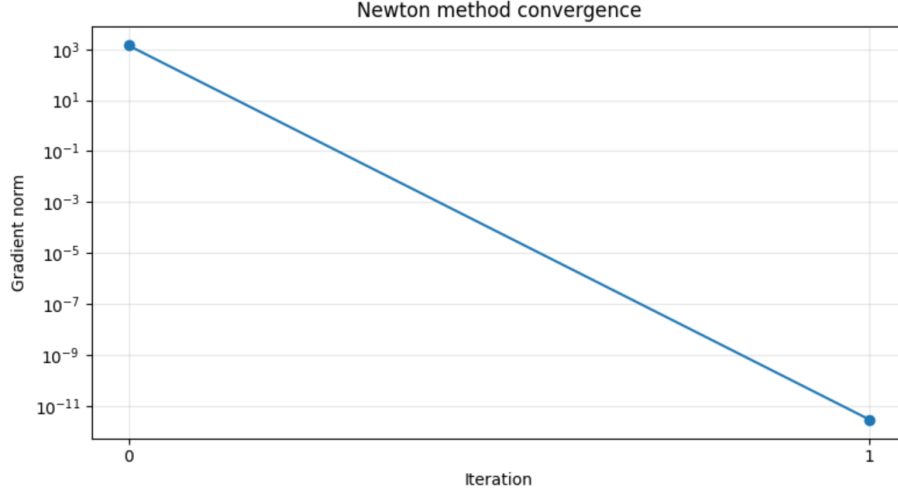


Figure 3.2: Convergence of Newton's method: gradient norm before and after one iteration.

3.9 Linear Conjugate Gradient

Theorem 3.2 (Finite convergence of conjugate gradient). *Assume that A is symmetric positive definite. In exact arithmetic, the conjugate gradient method applied to*

$$Ax = b$$

converges to the exact solution in at most d iterations, where d is the dimension of the system.

Remark 3.3. In this project, the reduced dimension is $d = n - 1$. Since $n = 12$, we have $d = 11$. Therefore, in exact arithmetic, conjugate gradient should converge in at most 11 iterations. The observed number may be slightly larger due to numerical errors.

3.10 Linear system formulation

The first-order optimality condition is

$$\nabla \tilde{J}(x^*) = a + 2Bx^* = 0.$$

Equivalently,

$$Ax^* = b, \quad A = 2B, \quad b = -a.$$

3.11 Conjugate gradient algorithm

Starting from x_0 , define

$$r_0 = b - Ax_0, \quad p_0 = r_0.$$

Then iterate

$$\alpha_k = \frac{r_k^\top r_k}{p_k^\top A p_k}, \quad x_{k+1} = x_k + \alpha_k p_k,$$

$$r_{k+1} = r_k - \alpha_k A p_k, \quad \beta_{k+1} = \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}, \quad p_{k+1} = r_{k+1} + \beta_{k+1} p_k.$$

The residual is linked to the gradient by

$$Ax_k - b = \nabla \tilde{J}(x_k).$$

Therefore, controlling the residual norm is exactly equivalent to controlling the gradient norm.

3.12 Numerical result for $n = 12$ and $\phi = 5$

For $n = 12$, the reduced dimension is 11. In exact arithmetic, CG terminates in at most 11 iterations, but in floating-point arithmetic more iterations may be needed because of rounding errors and loss of conjugacy.

The Python implementation gives:

- iterations: 14,
- final residual norm: approximately 2.242×10^{-8} ,
- objective value: $\tilde{J}(x^*) \approx 3.041284972985$,
- gradient norm: approximately 2.242×10^{-8} .

The reduced minimizer is approximately

$$x^* = \begin{pmatrix} 0.2685103513 \\ -0.6015878450 \\ -0.1865133437 \\ 0.9501440257 \\ 0.0579752600 \\ -0.0142005760 \\ -0.3957278065 \\ 0.1801993260 \\ 0.2418331666 \\ 0.4186345470 \\ 0.0642610830 \end{pmatrix}, \quad x_{12} \approx 0.0164718118.$$

This agrees with Newton's method and shows a much faster convergence than steepest descent.

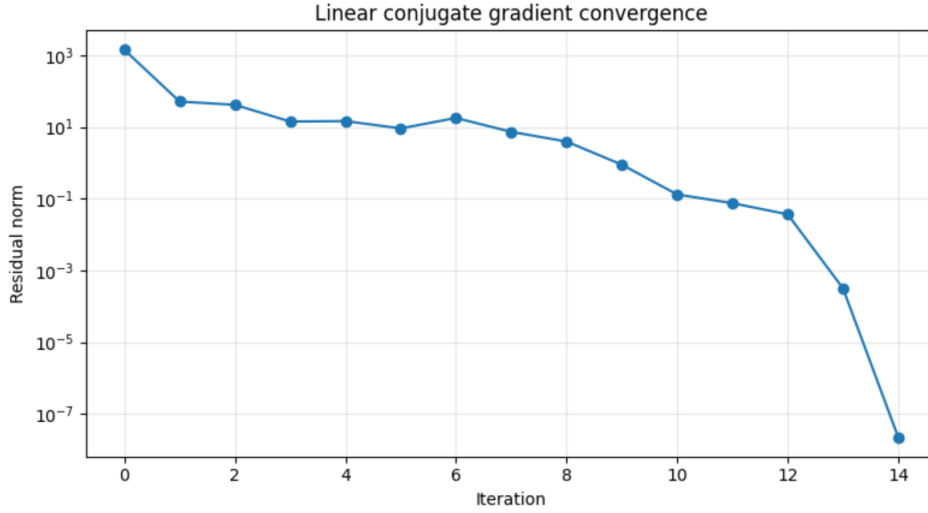


Figure 3.3: Convergence of the linear conjugate gradient method: residual norm versus iteration.

3.13 Solution Using an Optimization Library

Proposition 3.3 (Agreement of unconstrained solvers). *All unconstrained methods considered in Questions 2–5 solve the same first-order optimality condition*

$$\nabla \tilde{J}(x) = 0.$$

Therefore, up to numerical tolerance, they must converge to the same minimizer when the quadratic objective is strictly convex.

Proof. The reduced objective is quadratic:

$$\tilde{J}(x) = c + a^\top x + x^\top Bx.$$

Its first-order optimality condition is

$$\nabla \tilde{J}(x) = a + 2Bx = 0.$$

If B is positive definite, this equation admits a unique solution. Steepest descent, Newton’s method, conjugate gradient and the SciPy solvers all aim to solve this same condition. Hence, when they converge, they return the same minimizer up to numerical tolerance. $\square$

3.14 Reference solution

A reference solution is obtained by solving the linear system

$$(2B)x = -a.$$

For the project data,

$$\tilde{J}(x_{\text{ref}}) \approx 3.041284972984, \quad \|\nabla \tilde{J}(x_{\text{ref}})\| \approx 1.819 \times 10^{-12}.$$

3.15 SciPy BFGS

Using `scipy.optimize.minimize` with BFGS gives approximately

$$\begin{aligned}\text{success} &= \text{False}, & \text{iterations} &= 16, \\ \tilde{J}(x^*) &\approx 3.041284972984, \\ \|\nabla \tilde{J}(x^*)\| &\approx 1.309 \times 10^{-7}, & \|x^* - x_{\text{ref}}\| &\approx 2.864 \times 10^{-9}.\end{aligned}$$

Although the solver returns `success = False`, the computed point is still extremely close to the reference solution. This can happen when the stopping criterion of the solver is stricter than the numerical precision effectively reached by the algorithm.

3.16 SciPy Newton-CG

Using `Newton-CG` gives approximately

$$\begin{aligned}\text{success} &= \text{True}, & \text{iterations} &= 14, \\ \tilde{J}(x^*) &\approx 3.041284972984, \\ \|\nabla \tilde{J}(x^*)\| &\approx 1.205 \times 10^{-9}, & \|x^* - x_{\text{ref}}\| &\approx 3.144 \times 10^{-11}.\end{aligned}$$

3.17 Discussion

Both library solvers return a point that is numerically very close to the reference solution. Newton-CG reports successful convergence, while BFGS does not satisfy its internal success criterion. However, the BFGS solution remains accurate, as shown by the very small distance to the reference solution and the objective value, which matches the reference value up to numerical precision.

Chapter 4: Constrained Portfolio Optimization

Definition 4.1 (Simplex). The unit simplex in $\mathbb{R}^n$ is defined by

$$\Delta_n = \left\{ x \in \mathbb{R}^n : x_i \geq 0, \sum_{i=1}^n x_i = 1 \right\}.$$

In portfolio optimization, this set represents all admissible portfolios without short-selling.

Proposition 4.1 (Convexity of the constrained problem). *If the covariance matrix Σ is positive semidefinite, then the constrained portfolio problem*

$$\min_{x \in \Delta_n} x^\top \Sigma x - \phi \mu^\top x$$

is a convex optimization problem.

Proof. The Hessian of the objective function is

$$\nabla^2 J(x) = 2\Sigma.$$

If Σ is positive semidefinite, then 2Σ is also positive semidefinite. Therefore, J is convex. Moreover, the simplex Δ_n is a convex set because it is defined by linear equality and inequality constraints. Hence the problem is convex. $\square$

4.1 Simplex-constrained problem

We now return to the constrained problem

$$\min_{x \in \mathbb{R}^n} J(x) = x^\top \Sigma x - \phi \mu^\top x \quad \text{subject to} \quad x \geq 0, \quad \mathbf{1}^\top x = 1.$$

This is a convex quadratic program on the simplex.

4.2 KKT system on the free set

Let A be the active set and F the free set:

$$A = \{i : x_i = 0\}, \quad F = \{1, \dots, n\} \setminus A.$$

On the free set, stationarity gives

$$2\Sigma_{FF}x_F - \phi\mu_F + \lambda\mathbf{1}_F = 0, \quad \mathbf{1}_F^\top x_F = 1.$$

Thus we solve the KKT system

$$\begin{pmatrix} 2\Sigma_{FF} & \mathbf{1}_F \\ \mathbf{1}_F^\top & 0 \end{pmatrix} \begin{pmatrix} x_F \\ \lambda \end{pmatrix} = \begin{pmatrix} \phi\mu_F \\ 1 \end{pmatrix}.$$

Definition 4.2 (KKT conditions). For the constrained problem

$$\min_{x \in \mathbb{R}^n} J(x) \quad \text{subject to} \quad x_i \geq 0, \quad \mathbf{1}^\top x = 1,$$

the Karush–Kuhn–Tucker conditions combine feasibility, stationarity and complementarity. They characterize the optimum when the problem is convex.

Proposition 4.2 (KKT characterization on the simplex). *Let x^* be an optimal solution and let λ be the Lagrange multiplier associated with the equality constraint. Then there exist multipliers $\nu_i \geq 0$ such that*

$$\begin{aligned} 2\Sigma x^* - \phi\mu + \lambda\mathbf{1} - \nu &= 0, \\ \mathbf{1}^\top x^* &= 1, \quad x_i^* \geq 0, \quad \nu_i x_i^* = 0 \quad \text{for all } i. \end{aligned}$$

4.3 SLSQP implementation

In the Python file, the constrained problem is also solved using SLSQP from `scipy.optimize.minimize`. This method handles directly:

- the equality constraint $\mathbf{1}^\top x = 1$,
- the bound constraints $x_i \geq 0$.

4.4 Numerical optimum for $n = 12$ and $\phi = 5$

The constrained optimization problem is successfully solved by SLSQP. The solver returns:

$$\text{success} = \text{True}, \quad \text{iterations} = 15.$$

The constrained optimum is approximately

$$x^* = \begin{pmatrix} 0.0954662028 \\ 0 \\ 0 \\ 0.2719349423 \\ 0.0324107050 \\ 0 \\ 0 \\ 0.0351558043 \\ 0.2025676264 \\ 0.3434857758 \\ 0.0189789434 \\ 0 \end{pmatrix}.$$

Moreover,

$$\sum_{i=1}^{12} x_i^* \approx 1.0000000000000027, \quad \min_i x_i^* = 0.$$

The objective value is

$$J(x^*) \approx 7.918837100810942.$$

Compared with the unconstrained solution, several weights are exactly zero. Therefore, the constrained optimum lies on the boundary of the simplex. This illustrates the effect of the no-short-selling constraint.

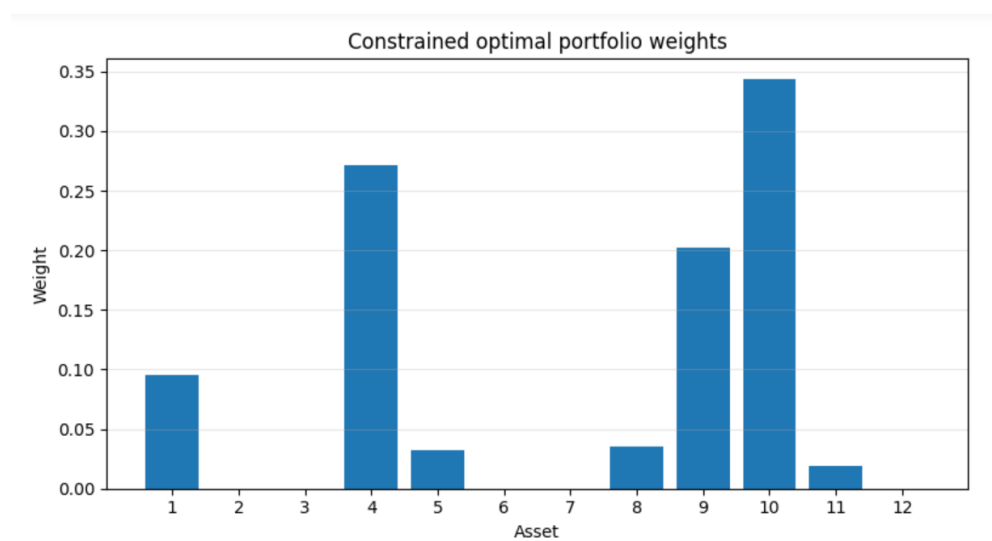


Figure 4.1: Optimal constrained portfolio weights.

Conclusion

This project allowed us to study a classical mean–variance portfolio optimization problem from both a theoretical and numerical point of view. After formulating the original constrained problem, we first eliminated the equality constraint in order to obtain a reduced problem in dimension $n - 1$. This transformation made it possible to study the objective as a quadratic function and to apply several unconstrained optimization methods.

The numerical experiments show that all unconstrained methods converge to the same minimizer up to numerical precision. However, their convergence speeds are very different. Steepest descent requires many iterations because it only uses first-order information and is sensitive to the conditioning of the quadratic form. Newton’s method, on the other hand, converges in one iteration because the objective function is quadratic and its Hessian is constant. The linear conjugate gradient method also performs very efficiently, since it is well adapted to the linear system associated with the first-order optimality condition.

The comparison with optimization routines from `scipy.optimize` confirms the consistency of the results obtained manually. These solvers return points that are numerically very close to the reference solution, which validates the implementation of the previous methods.

Finally, enforcing the no-short-selling constraints changes the nature of the solution. The constrained optimum belongs to the simplex and has several zero components, meaning that some assets are excluded from the optimal allocation. This illustrates the practical importance of constraints in portfolio optimization: even when the unconstrained solution is mathematically optimal, it may not be financially admissible.

Overall, this project highlights the importance of choosing an appropriate numerical method. For quadratic problems, second-order methods and Krylov methods are much more efficient than basic gradient descent, while constrained problems require dedicated algorithms capable of preserving feasibility.

Appendix: Python Code

The full Python code extracted from the notebook is included below.

```
1 # =====
2 # Numerical Methods Project
3 # Mean--Variance Portfolio Optimization
4 #
5 # This file contains:
6 # - the full numerical program for Questions 2 to 6,
7 # - the generation of all figures used in the LaTeX report,
8 # - no duplicated function definitions.
9 #
10 # Generated figures:
11 # figures/steepest_descent_convergence.png
12 # figures/newton_convergence.png
13 # figures/conjugate_gradient_convergence.png
14 # figures/constrained_portfolio.png
15 # =====
16
17 import os
18 import numpy as np
19 import numpy.linalg as la
20 import matplotlib.pyplot as plt
21 from scipy.optimize import minimize
22
23 # =====
24 # GLOBAL SETUP
25 # =====
26
27 np.set_printoptions(precision=10, suppress=True)
28 os.makedirs("figures", exist_ok=True)
29
30 phi = 5.0
31 n = 12
32
33 mu = np.array(
34     [3.66, 1.33, 2.51, 2.24, 2.62, 2.03, 1.91, 1.39, 2.61, 3.58, 5.23, 8.36],
35     dtype=float,
36 )
37
38 Sigma = np.array(
39     [
40         [73.73, 40.79, 39.46, 38.01, 49.40, 58.51, 46.08, 42.26, -1.22, 12.44, -2.31, 131.17],
41         [40.79, 39.54, 32.15, 34.43, 34.25, 36.88, 33.11, 26.50, -3.13, 19.06, 0.53, 136.78],
42         [39.46, 32.15, 54.98, 31.62, 33.80, 41.20, 26.48, 29.76, -2.90, 25.34, -3.62, 151.82],
43         [38.01, 34.43, 31.62, 39.61, 35.73, 33.64, 38.12, 26.00, -0.08, 13.23, -5.61, 73.96],
44         [49.40, 34.25, 33.80, 35.73, 68.10, 36.88, 37.02, 31.64, 14.42, 3.33, 1.43, -0.12],
45         [58.51, 36.88, 41.20, 33.64, 36.88, 66.93, 40.89, 39.08, -13.71, 22.07, -14.96, 210.26],
46         [46.08, 33.11, 26.48, 38.12, 37.02, 40.89, 75.40, 38.58, 19.01, 19.37, 30.95, 52.63],
47         [42.26, 26.50, 29.76, 26.00, 31.64, 39.08, 38.58, 47.35, -3.53, 17.02, -8.28, 111.37],
48         [-1.22, -3.13, -2.90, -0.08, 14.42, -13.71, 19.01, -3.53, 242.03, -90.65, 196.18, -507.15],
49         [12.44, 19.06, 25.34, 13.23, 3.33, 22.07, 19.37, 17.02, -90.65, 110.87, -50.97, 360.36],
50         [-2.31, 0.53, -3.62, -5.61, 1.43, -14.96, 30.95, -8.28, 196.18, -50.97, 428.23, -504.38],
51         [131.17, 136.78, 151.82, 73.96, -0.12, 210.26, 52.63, 111.37, -507.15, 360.36, -504.38, 2863.07],
52     ],
53     dtype=float,
54 )
55
56 # =====
57 # REDUCED QUADRATIC PROBLEM
58 # =====
59
60 Cov_nn = Sigma[n - 1, n - 1]
```

```

61 cov_in = Sigma[: n - 1, n - 1]
62
63 a = -phi * (mu[: n - 1] - mu[n - 1]) - 2.0 * Cov_nn + 2.0 * cov_in
64 B = Sigma[: n - 1, : n - 1] - cov_in[:, None] - cov_in[None, :] + Cov_nn
65 const = -phi * mu[n - 1] + Cov_nn
66
67 H = 2.0 * B
68 A = H
69 b = -a
70
71
72 def Je(x: np.ndarray) -> float:
73     """Reduced objective function in  $R^{n-1}$ ."""
74     x = np.asarray(x, dtype=float).reshape(-1)
75     return float(const + x @ a + x @ (B @ x))
76
77
78 def grad_Je(x: np.ndarray) -> np.ndarray:
79     """Gradient of the reduced objective:  $a + 2Bx$ ."""
80     x = np.asarray(x, dtype=float).reshape(-1)
81     return a + 2.0 * (B @ x)
82
83
84 def J(x: np.ndarray) -> float:
85     """Full objective function in  $R^n$ :  $x^T \text{Sigma } x - \text{phi } \mu^T x$ ."""
86     x = np.asarray(x, dtype=float).reshape(-1)
87     return float(x @ Sigma @ x - phi * (mu @ x))
88
89
90 def gradJ(x: np.ndarray) -> np.ndarray:
91     """Gradient of the full objective function."""
92     x = np.asarray(x, dtype=float).reshape(-1)
93     return 2.0 * Sigma @ x - phi * mu
94
95 # =====
96 # QUESTION 2 - STEEPEST DESCENT
97 # =====
98
99 def steepest_descent_exact_wolfe(x0, tol=1e-5, max_iter=5_000_000, store_history=False):
100     """
101     Steepest descent with exact line search.
102
103     For a strictly convex quadratic objective, the exact line-search step
104     is Wolfe-compliant. If store_history=True, the function also returns
105     the gradient norm and objective histories for plotting.
106     """
107     x = np.asarray(x0, dtype=float).reshape(-1).copy()
108
109     grad_norm_history = []
110     objective_history = []
111
112     for k in range(max_iter):
113         g = grad_Je(x)
114         grad_norm = float(la.norm(g))
115
116         if store_history:
117             grad_norm_history.append(grad_norm)
118             objective_history.append(Je(x))
119
120         if grad_norm <= tol:
121             if store_history:
122                 return x, Je(x), k, grad_norm, grad_norm_history, objective_history
123             return x, Je(x), k, grad_norm
124
125         denom = float(g @ (H @ g))
126
127         if denom <= 0:
128             alpha = 1e-3

```

```

129         else:
130             alpha = float((g @ g) / denom)
131
132             x = x - alpha * g
133
134         final_grad_norm = float(la.norm(grad_Je(x)))
135
136         if store_history:
137             return x, Je(x), max_iter, final_grad_norm, grad_norm_history, objective_history
138         return x, Je(x), max_iter, final_grad_norm
139
140     # =====
141     # QUESTION 3 - NEWTON METHOD WITH WOLFE LINE SEARCH
142     # =====
143
144     def strong_wolfe_line_search(xk, pk, c1=1e-4, c2=0.9, alpha0=1.0, alpha_max=10.0, max_iter=50):
145         """Strong Wolfe line search for phi(alpha)=Je(xk+alpha pk)."""
146         xk = np.asarray(xk, dtype=float).reshape(-1)
147         pk = np.asarray(pk, dtype=float).reshape(-1)
148
149         def phi_alpha(alpha):
150             return Je(xk + alpha * pk)
151
152         def dphi_alpha(alpha):
153             return float(grad_Je(xk + alpha * pk) @ pk)
154
155         phi0 = phi_alpha(0.0)
156         dphi0 = dphi_alpha(0.0)
157
158         if dphi0 >= 0:
159             return 0.0
160
161         alpha_prev = 0.0
162         phi_prev = phi0
163         alpha = alpha0
164
165         def zoom(alo, ahi):
166             philo = phi_alpha(alo)
167             for _ in range(max_iter):
168                 aj = 0.5 * (alo + ahi)
169                 phij = phi_alpha(aj)
170
171                 if (phij > phi0 + c1 * aj * dphi0) or (phij >= philo):
172                     ahi = aj
173                 else:
174                     dphij = dphi_alpha(aj)
175                     if abs(dphij) <= -c2 * dphi0:
176                         return aj
177                     if dphij * (ahi - alo) >= 0:
178                         ahi = alo
179                     alo = aj
180                     philo = phij
181
182                 if abs(ahi - alo) < 1e-14:
183                     return aj
184             return aj
185
186         for _ in range(max_iter):
187             phia = phi_alpha(alpha)
188
189             if (phia > phi0 + c1 * alpha * dphi0) or (phia >= phi_prev and alpha > 0):
190                 return zoom(alpha_prev, alpha)
191
192             dphia = dphi_alpha(alpha)
193
194             if abs(dphia) <= -c2 * dphi0:
195                 return alpha

```

```

197         if dphia >= 0:
198             return zoom(alpha, alpha_prev)
199
200         alpha_prev = alpha
201         phi_prev = phia
202         alpha = min(2.0 * alpha, alpha_max)
203
204     return alpha
205
206
207 def newton_wolfe(x0, tol=1e-10, max_iter=50, store_history=False):
208     """
209     Newton method with Wolfe line search for the reduced quadratic problem.
210     If store_history=True, returns gradient norm history for plotting.
211     """
212     x = np.asarray(x0, dtype=float).reshape(-1).copy()
213     grad_norm_history = []
214
215     for k in range(max_iter):
216         g = grad_Je(x)
217         grad_norm = float(la.norm(g))
218
219         if store_history:
220             grad_norm_history.append(grad_norm)
221
222         if grad_norm <= tol:
223             if store_history:
224                 return x, Je(x), k, grad_norm_history
225             return x, Je(x), k
226
227         p = la.solve(H, -g)
228         alpha = strong_wolfe_line_search(x, p)
229
230         if alpha == 0.0:
231             alpha = 1.0
232
233         x = x + alpha * p
234
235         if store_history:
236             grad_norm_history.append(float(la.norm(grad_Je(x))))
237         return x, Je(x), max_iter, grad_norm_history
238
239     return x, Je(x), max_iter
240
241 # =====
242 # QUESTION 4 - LINEAR CONJUGATE GRADIENT
243 # =====
244
245 def conjugate_gradient(A, b, x0, tol=1e-10, max_iter=10_000, store_history=False):
246     """
247     Linear conjugate gradient method for Ax=b.
248     If store_history=True, also returns the residual norm history.
249     """
250     A = np.asarray(A, dtype=float)
251     b = np.asarray(b, dtype=float).reshape(-1)
252     x = np.asarray(x0, dtype=float).reshape(-1).copy()
253
254     r = A @ x - b
255     p = -r
256     rTr = float(r @ r)
257     r0_norm = float(np.sqrt(rTr))
258
259     residual_history = [float(la.norm(r))] if store_history else None
260
261     if r0_norm == 0.0:
262         if store_history:
263             return x, 0, 0.0, residual_history
264         return x, 0, 0.0

```

```

265
266     for k in range(max_iter):
267         Ap = A @ p
268         denom = float(p @ Ap)
269
270         if denom <= 0:
271             break
272
273         alpha = rTr / denom
274         x = x + alpha * p
275         r = r + alpha * Ap
276
277         rTr_new = float(r @ r)
278         r_norm = float(np.sqrt(rTr_new))
279
280         if store_history:
281             residual_history.append(r_norm)
282
283         if r_norm <= tol * r0_norm:
284             if store_history:
285                 return x, k + 1, r_norm, residual_history
286             return x, k + 1, r_norm
287
288         beta = rTr_new / rTr
289         p = -r + beta * p
290         rTr = rTr_new
291
292     final_residual_norm = float(la.norm(A @ x - b))
293
294     if store_history:
295         return x, k + 1, final_residual_norm, residual_history
296     return x, k + 1, final_residual_norm
297
298 # =====
299 # QUESTION 5 - SCIPY OPTIMIZATION LIBRARY
300 # =====
301
302 def solve_unconstrained_with_scipy(x0):
303     """Solve the reduced unconstrained problem using SciPy BFGS and Newton-CG."""
304     x0 = np.asarray(x0, dtype=float).reshape(-1)
305
306     x_ref = la.solve(H, -a)
307     f_ref = Je(x_ref)
308
309     res_bfgs = minimize(
310         fun=lambda z: Je(z),
311         x0=x0,
312         jac=lambda z: grad_Je(z),
313         method="BFGS",
314         options={"gtol": 1e-10, "disp": False},
315     )
316
317     res_ncg = minimize(
318         fun=lambda z: Je(z),
319         x0=x0,
320         jac=lambda z: grad_Je(z),
321         hessp=lambda z, v: H @ v,
322         method="Newton-CG",
323         options={"xtol": 1e-12, "disp": False, "maxiter": 50},
324     )
325
326     return x_ref, f_ref, res_bfgs, res_ncg
327
328 # =====
329 # QUESTION 6 - CONSTRAINED OPTIMIZATION ON THE SIMPLEX
330 # =====
331
332 def solve_q6_slsqp(x0=None, tol=1e-10, max_iter=10_000):

```

```

333 """
334 Solve the constrained portfolio problem:
335
336     min J(x)
337     subject to  $x_i \geq 0$  and  $\sum_i x_i = 1$ 
338
339 using SLSQP.
340 """
341 if x0 is None:
342     x0 = np.ones(n) / n
343
344 x0 = np.asarray(x0, dtype=float).reshape(-1)
345
346 constraints = (
347     {
348         "type": "eq",
349         "fun": lambda x: np.sum(x) - 1.0,
350         "jac": lambda x: np.ones_like(x),
351     },
352 )
353
354 bounds = [(0.0, None) for _ in range(n)]
355
356 res = minimize(
357     J,
358     x0,
359     jac=gradJ,
360     method="SLSQP",
361     constraints=constraints,
362     bounds=bounds,
363     options={"ftol": tol, "maxiter": max_iter, "disp": True},
364 )
365
366 return res
367
368 # =====
369 # FIGURE GENERATION
370 # =====
371
372 def save_steepest_descent_figure(grad_norm_history):
373     """Save the steepest descent convergence figure."""
374     plt.figure(figsize=(8, 4.5))
375     plt.semilogy(grad_norm_history)
376     plt.xlabel("Iteration")
377     plt.ylabel("Gradient norm")
378     plt.title("Steepest descent convergence")
379     plt.grid(True, which="both", alpha=0.3)
380     plt.tight_layout()
381     plt.savefig("figures/steepest_descent_convergence.png", dpi=300, bbox_inches="tight")
382     plt.close()
383
384
385 def save_newton_figure(grad_norm_history):
386     """Save the Newton convergence figure."""
387     x_axis = list(range(len(grad_norm_history)))
388     plt.figure(figsize=(8, 4.5))
389     plt.semilogy(x_axis, grad_norm_history, marker="o")
390     plt.xlabel("Iteration")
391     plt.ylabel("Gradient norm")
392     plt.title("Newton method convergence")
393     plt.xticks(x_axis)
394     plt.grid(True, which="both", alpha=0.3)
395     plt.tight_layout()
396     plt.savefig("figures/newton_convergence.png", dpi=300, bbox_inches="tight")
397     plt.close()
398
399
400 def save_conjugate_gradient_figure(residual_history):

```

```

401     """Save the conjugate gradient convergence figure."""
402     plt.figure(figsize=(8, 4.5))
403     plt.semilogy(residual_history, marker="o")
404     plt.xlabel("Iteration")
405     plt.ylabel("Residual norm")
406     plt.title("Linear conjugate gradient convergence")
407     plt.grid(True, which="both", alpha=0.3)
408     plt.tight_layout()
409     plt.savefig("figures/conjugate_gradient_convergence.png", dpi=300, bbox_inches="tight")
410     plt.close()
411
412
413 def save_constrained_portfolio_figure(x_constrained):
414     """Save the constrained optimal portfolio weights figure."""
415     x_plot = np.asarray(x_constrained, dtype=float).reshape(-1)
416     plt.figure(figsize=(8, 4.5))
417     plt.bar(np.arange(1, len(x_plot) + 1), x_plot)
418     plt.xlabel("Asset")
419     plt.ylabel("Weight")
420     plt.title("Constrained optimal portfolio weights")
421     plt.xticks(np.arange(1, len(x_plot) + 1))
422     plt.grid(axis="y", alpha=0.3)
423     plt.tight_layout()
424     plt.savefig("figures/constrained_portfolio.png", dpi=300, bbox_inches="tight")
425     plt.close()
426
427 # =====
428 # MAIN EXECUTION
429 # =====
430
431 def main():
432     x0_reduced = np.ones(n - 1) * (1.0 / n)
433
434     # Question 2
435     x_sd, f_sd, it_sd, grad_sd, grad_hist_sd, obj_hist_sd = steepest_descent_exact_wolfe(
436         x0_reduced,
437         tol=1e-5,
438         store_history=True,
439     )
440     x12_sd = 1.0 - np.sum(x_sd)
441
442     print("=== Question 2: Steepest descent (exact/Wolfe-compliant line search) ===")
443     print(f"Iterations: {it_sd}")
444     print(f"||grad|| at stop: {grad_sd:.6e}")
445     print(f"x* (reduced, length n-1):")
446     print(x_sd.reshape(n - 1, 1))
447     print(f"Implied x_n = 1 - sum(x_i) = {x12_sd:.8f}")
448     print(f"J_tilde(x*) = {f_sd:.12f}")
449     print()
450
451     save_steepest_descent_figure(grad_hist_sd)
452
453     # Question 3
454     x_newton, f_newton, it_newton, grad_hist_newton = newton_wolfe(
455         x0_reduced,
456         tol=1e-10,
457         max_iter=50,
458         store_history=True,
459     )
460     x12_newton = 1.0 - np.sum(x_newton)
461
462     print("==== Question 3: Newton + Wolfe ====")
463     print(f"Iterations: {it_newton}")
464     print(f"x* (reduced):")
465     print(x_newton.reshape(n - 1, 1))
466     print(f"Implied x_n = {x12_newton:.8f}")
467     print(f"J_tilde(x*) = {f_newton:.12f}")
468     print()

```

```

469
470 save_newton_figure(grad_hist_newton)
471
472 # Question 4
473 x_cg, it_cg, res_norm_cg, residual_hist_cg = conjugate_gradient(
474     A,
475     b,
476     x0_reduced,
477     tol=1e-10,
478     max_iter=5000,
479     store_history=True,
480 )
481 x12_cg = 1.0 - np.sum(x_cg)
482
483 print("==== Question 4: Linear Conjugate Gradient (unconstrained) ====")
484 print(f"Iterations: {it_cg}")
485 print(f"Final residual norm ||Ax-b||: {res_norm_cg:.3e}")
486 print("x* (reduced):")
487 print(x_cg.reshape(n - 1, 1))
488 print(f"Implied x_n = {x12_cg:.10f}")
489 print(f"J_tilde(x*) = {Je(x_cg):.12f}")
490 print(f"||grad J(x*)|| = {la.norm(grad_Je(x_cg)):.3e}")
491 print()
492
493 save_conjugate_gradient_figure(residual_hist_cg)
494
495 # Question 5
496 x_ref, f_ref, res_bfgs, res_ncg = solve_unconstrained_with_scipy(x0_reduced)
497
498 print("==== Question 5 ====")
499 print("==== Reference (linear solve) ====")
500 print(f"J_tilde(x_ref) = {f_ref:.12f}")
501 print(f"||grad|| = {la.norm(grad_Je(x_ref)):.3e}")
502 print()
503
504 print("==== SciPy minimize: BFGS ====")
505 print(f"success: {res_bfgs.success}, iters: {res_bfgs.nit}")
506 print(f"J_tilde(x*) = {res_bfgs.fun:.12f}")
507 print(f"||grad|| = {la.norm(res_bfgs.jac):.3e}")
508 print(f"||x*-x_ref|| = {la.norm(res_bfgs.x - x_ref):.3e}")
509 print()
510
511 print("==== SciPy minimize: Newton-CG ====")
512 print(f"success: {res_ncg.success}, iters: {res_ncg.nit}")
513 print(f"J_tilde(x*) = {res_ncg.fun:.12f}")
514 print(f"||grad|| = {la.norm(res_ncg.jac):.3e}")
515 print(f"||x*-x_ref|| = {la.norm(res_ncg.x - x_ref):.3e}")
516 print()
517
518 # Question 6
519 print("==== Question 6 ====")
520 res_q6 = solve_q6_slsqp()
521
522 x_q6 = res_q6.x
523 J_q6 = J(x_q6)
524
525 print("Success:", res_q6.success)
526 print("Message:", res_q6.message)
527 print("Iterations:", res_q6.nit)
528 print("sum(x*) =", np.sum(x_q6))
529 print("min(x*) =", np.min(x_q6))
530 print("J(x*) =", J_q6)
531 print("x* =")
532 print(x_q6.reshape(n, 1))
533
534 save_constrained_portfolio_figure(x_q6)
535
536 print()

```

```
537     print("Figures saved in the folder: figures/")
538     print(" - figures/steepest_descent_convergence.png")
539     print(" - figures/newton_convergence.png")
540     print(" - figures/conjugate_gradient_convergence.png")
541     print(" - figures/constrained_portfolio.png")
542
543
544 if __name__ == "__main__":
545     main()
```

Listing 1: Project Python code